INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 1

Assignment 1: Reading Array Values

Assignment aim: Read and process array values using indexes and loops.

Experiment No. 1.1 — Read first and last array values

1. Experiment Title: Read first and last array values

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Arrays store same-type values in indexed positions; loops allow repeated access to their elements.

5. Algorithm / Procedure:
Create the array; access index 0 and index length - 1; print both values.
Then compile and execute the Java program.

6. Program:

```
public class Task_01_Read_first_and_last_array_values{
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        System.out.println("First: " + marks[0]);
        System.out.println("Last: " + marks[marks.length - 1]);
    }
}
```

7. Input: Fixed input values are declared in the program: marks = {45, 67, 80, 52, 90}.

8. Output:
First: 45
Last: 90

Time / Space Complexity: O(1) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is an array index?
- What is the first valid index in Java?
- How do you find the last index?

Experiment No. 1.2 — Print all array values

1. Experiment Title: Print all array values

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Arrays store same-type values in indexed positions; loops allow repeated access to their elements.

5. Algorithm / Procedure:
Initialize the array; loop from 0 to length - 1; print each element.
Then compile and execute the Java program.

6. Program:

```
public class Task_02_Print_all_array_values{
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
        }
    }
}
```

7. Input: Fixed input values are declared in the program: marks = {45, 67, 80, 52, 90}.

8. Output:
45
67
80
52
90

Time / Space Complexity: O(n) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What does marks.length return?
- Why does the loop start at 0?
- What is a loop iteration?

Experiment No. 1.3 — Calculate array sum

1. Experiment Title: Calculate array sum

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Arrays store same-type values in indexed positions; loops allow repeated access to their elements.

5. Algorithm / Procedure:
Initialize sum to zero; visit every element; add each value to sum; print sum.
Then compile and execute the Java program.

6. Program / Queries / Commands:

```
public class Task_03_Calculate_array_sum{
    public static void main(String[] args) {
        int[] marks = {45, 67, 80, 52, 90};
        int sum = 0;
        for (int value : marks) {
            sum += value;
        }
        System.out.println("Sum: " + sum);
    }
}
```

7. Input: Fixed input values are declared in the program: marks = {45, 67, 80, 52, 90}.

8. Output:
Sum: 334

Time / Space Complexity: O(n) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is an accumulator?
- What is an enhanced for loop?
- Why initialize sum to 0?